TCP Handshake, FIN/RST & TIME_WAIT

how connections open, close, and run you out of ports

Watch the 3-min video

> **In one line:** a TCP connection costs a SYN, SYN-ACK, ACK round trip to open and a FIN/ACK exchange per direction to close, and the side that closes first holds its ports in TIME_WAIT for about 60 s, so many short-lived connections to one destination can exhaust the ephemeral ports.

1 The mystery: "cannot assign requested address"

Your service opens a fresh connection to a partner API for every request. Traffic grows, and calls start failing with `EADDRNOTAVAIL: cannot assign requested address`. CPU is fine, memory is fine. You have run out of **ports**. To see why, you need to know how TCP connections open and, more importantly, how they close.

2 Opening: the three-way handshake

Before any data flows, TCP does a **three-way handshake**:

1. The client sends **SYN** with its starting sequence number *x* (chosen at random).
2. The server replies **SYN-ACK**: its own starting number *y*, plus `ack = x + 1`, acknowledging the client's SYN.
3. The client sends **ACK** (`ack = y + 1`). The connection is open.

That costs **one full round trip** before the first byte of request data. At 80 ms RTT, every new connection adds 80 ms (and TLS adds more on top).

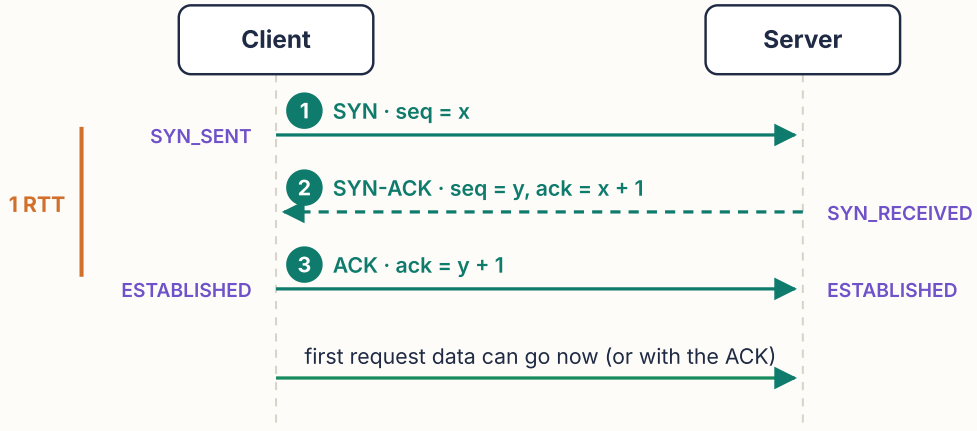
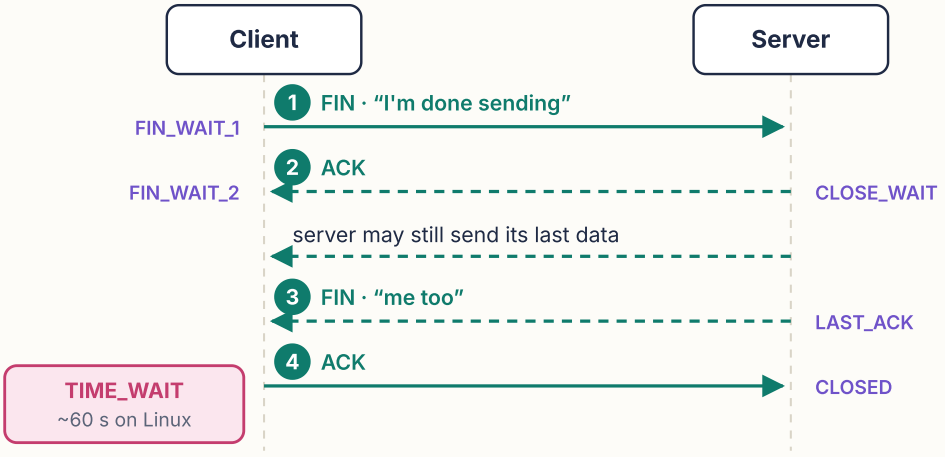


Figure 1. The three-way handshake. Both sides learn each other's starting sequence number; the client can send data right after its ACK.

3 Closing: FIN, ACK, and the abrupt RST

Closing takes four steps, because **each direction closes separately**. One side sends **FIN**: "I'm done sending." The other side ACKs it, finishes sending whatever it still has, then sends its own FIN, which is acknowledged too. (The middle two messages are often combined into one FIN-ACK.)

Sometimes there is no polite goodbye. A **RST** (reset) kills the connection instantly. You get one when you connect to a port nobody is listening on ("connection refused"), when a process crashes or closes a socket with unread data, or when a firewall or load balancer gives up on a connection.



No polite goodbye: a RST resets the connection at once (e.g. SYN to a port with no listener → "connection refused").

Figure 2. The four-way close, with the state each side passes through. The side that sent the first FIN ends in TIME_WAIT.

4 TIME_WAIT and the port math

The side that closes first enters **TIME_WAIT**. It keeps that connection's exact addresses and ports reserved for a while, **about 60 seconds on Linux**. Two reasons: a delayed packet from the old connection must not be mistaken for part of a new one, and if the final ACK is lost, the other side can resend its FIN and still get an answer.

Now the math. Every outgoing connection uses an **ephemeral port** on the client. Linux's default range is 32768–60999, about **28,000 ports**. A connection is identified by source IP, source port, destination IP and destination port, so the limit applies *per destination*:

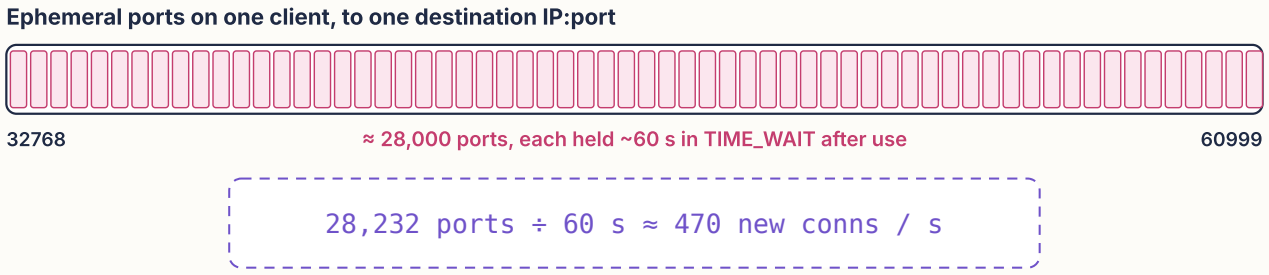


Figure 3. If every connection to the same server is closed by the client after one request, ports return to the pool only after 60 s, capping new connections at about 470 per second.

A **NAT gateway** hits the same wall: every instance behind it shares the gateway's ports for each destination, so a fleet of 50 servers calling one partner API through one NAT IP shares the same budget. (AWS NAT Gateway allows about 55,000 simultaneous connections per destination, per IP address.)

5 Half-open connections

Another failure mode: one side crashes, or a firewall or NAT silently drops an idle connection from its table. The other side still believes the connection is open. It only finds out when it sends data and gets a RST back, or no reply at all until its retransmissions give up, which can take many minutes.

TCP keepalives (an empty probe after a period of idleness; the Linux default waits 2 hours, so tune it to minutes) or **application heartbeats** (ping/pong messages) detect these dead connections early. Set the interval shorter than the idle timeout of any firewall, NAT or load balancer on the path, often 350 s for AWS NAT and NLB.

6 The fix: reuse connections

With **HTTP keep-alive** and **connection pools**, thousands of requests share a few long-lived connections. That removes the handshake cost and the TIME_WAIT pile-up together. If you still need more, spread traffic across **more IP addresses** (more source IPs, or more destination IPs), each with its own 28,000 ports. On Linux, `net.ipv4.tcp_tw_reuse` lets new *outgoing* connections safely reuse ports stuck in TIME_WAIT (it relies on TCP timestamps).

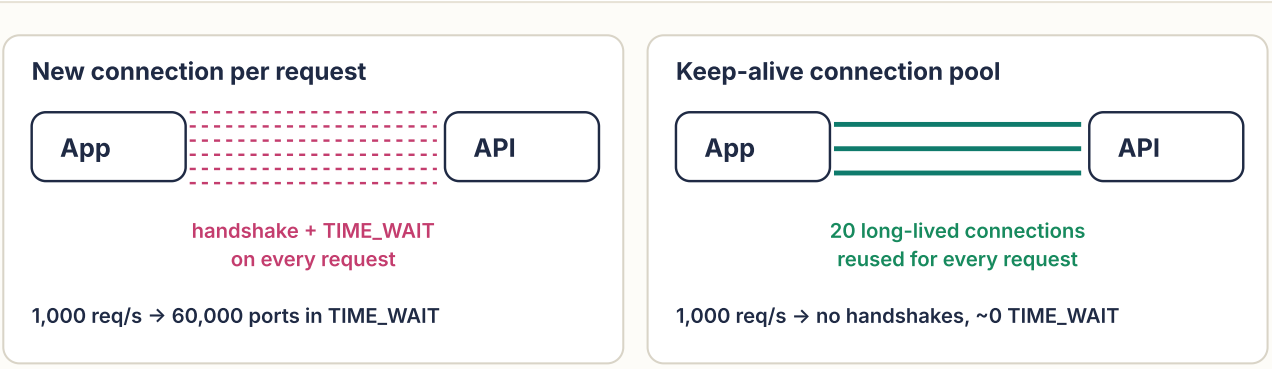


Figure 4. Left: a connection per request pays a handshake each time and leaves a port in TIME_WAIT. Right: a pool of warm connections serves the same load with none of that.

WORKED EXAMPLE

Your service makes **1,000 calls/s** to one partner endpoint, each on a new connection that your side closes. Ports in TIME_WAIT at steady state: 1,000 × 60 s = **60,000**, but only about 28,000 exist. Failures start at about 470 calls/s.

Switch to a keep-alive pool. By Little's Law, 1,000 calls/s at 20 ms each need about 1,000 × 0.02 = **20** connections in use at once. A pool of 30–50 covers bursts, with zero handshakes per call.

COMMON MISTAKES

- Creating a new HTTP client (and connection) per request instead of sharing one.
- Treating many TIME_WAIT sockets on a busy client as a leak. It's normal; it only hurts when it runs out the ports to one destination.
- Piles of **CLOSE_WAIT**, on the other hand, are a bug: the peer closed but your code never closed its socket.
- Turning on the old `tcp_tw_recycle`: it broke clients behind NAT and was removed from Linux in 4.12.

7 Key terms

SYN / SYN-ACK / ACK: the three-way handshake

FIN: “I'm done sending” for one direction

RST: abrupt reset of a connection

TIME_WAIT: closer holds the 4-tuple ~60 s

CLOSE_WAIT: peer closed; local app hasn't yet

Ephemeral port: temporary client-side source port

4-tuple: src IP, src port, dst IP, dst port

Half-open: one side gone, the other thinks it's open

Keepalive: idle probe that detects dead peers

8 Interview questions

Q1. Explain the TCP three-way handshake. BASIC

Client sends SYN with its starting sequence number; server replies SYN-ACK with its own number and an ACK of the client's; client sends ACK. It costs one round trip before data can flow, and both sides now know each other's sequence numbers.

Q2. Why does closing take four messages? BASIC

Each direction closes on its own. One side sends FIN and gets an ACK; the other side can keep sending, then sends its own FIN and gets an ACK. The middle ACK and FIN are often combined.

Q3. Why can TIME_WAIT cause port exhaustion? BASIC

The closing side keeps each connection's 4-tuple reserved for about 60 s. With about 28,000 ephemeral ports per destination, a client can open only about 470 new connections per second to the same server before ports run out.

Q4. Calls to a partner API fail with “cannot assign requested address” at peak, CPU is low. Diagnose and fix. SCENARIO

Port exhaustion: a new connection per call, closed by your side, leaves ports in TIME_WAIT. Check with ss -s or ss -tan state time-wait. Fix with a shared keep-alive pool; then add source IPs or tcp_tw_reuse if needed.

Q5. Users on a long-lived connection stop getting messages after 10 idle minutes, with no error. Why? SCENARIO

A firewall, NAT or load balancer dropped the idle connection (AWS NAT and NLB: 350 s), leaving it half-open. Send keepalives or app heartbeats every minute or so, under the shortest idle timeout on the path, and reconnect on failure.

Q6. A server shows thousands of sockets in CLOSE_WAIT. What's wrong? SCENARIO

The clients closed their side, but the application never called close(), usually a leaked connection on an error path. It's an application bug; restart buys time, the code needs fixing.

Q7. Which side ends up in TIME_WAIT, and how can a server avoid holding them? TRICKY

Whichever side sends the first FIN. Servers usually let clients close idle keep-alive connections, so TIME_WAIT spreads across many client machines. A busy server rarely runs out of ports anyway: every connection has a different client IP or port, so its 4-tuples stay unique; TIME_WAIT only costs it a little memory.

Q8. Why not just set TIME_WAIT to 1 second? TRICKY

It exists so stray delayed packets from an old connection can't corrupt a new one with the same 4-tuple, and so a lost final ACK can be recovered. On Linux it's fixed at 60 s; tcp_tw_reuse (with timestamps) is the safe way to reuse ports for outgoing connections.

Next: SD-B-005 · UDP vs TCP backendroadmap.com